

An Approximate and Iterative Posit Multiplier Architecture for FPGAs

Cameron James Norris and Sunwoong Kim

Division of Engineering and Mathematics, University of Washington, Bothell, WA 98011, USA

Email: {cjin0827, sunwoong}@uw.edu

Abstract—Recently, many applications have demanded cheaper and faster arithmetic while providing a wider dynamic range than the popular IEEE 754 floating-point (FP) arithmetic. As a result, a new number format called posit was proposed. As in a variety of number systems, multiplication in posit arithmetic is one of the most frequently used but expensive operations. To reduce the number of hardware resources in a posit multiplier, this paper applies an iterative approach to posit multiplication. To exploit the features of the posit format, the number of truncated bits in the fraction component is dynamically changed depending on the number of regime bits. In addition, architectures for fast parser and packer are proposed. Thanks to the posit format, the proposed design supports about 60 decades wider dynamic range than a single-precision FP multiplier design. Using the iterative approach, the proposed design also reduces the number of look-up tables used in a previous exact posit multiplier design by 44% while achieving a 51% higher maximum frequency, and the appropriate balance between latency and accuracy can be finely determined at runtime. To the best of the authors' knowledge, this paper is the first work that proposes a hardware architecture of an approximate and iterative posit multiplier.

I. INTRODUCTION

Floating-point (FP) arithmetic is essential in many applications using real numbers. For FP arithmetic, the standard IEEE 754 formats have been used over 30 years [1]. The most popular IEEE 754 formats are 32-bit single-precision FP (SPFP) format and 64-bit double-precision FP (DPFP) format, which are commonly used in high-level programming languages. Despite their firm establishment in arithmetic with real numbers, recent application areas such as machine learning (ML) have demanded cheaper and faster arithmetic providing wider dynamic range and/or higher accuracy as well [2].

As a result, a new format called *universal number (unum)* was first proposed by Gustafson in 2015 [3]. Since its first appearance, it has evolved to the present type-3 unum, called *posit* [4]. The posit has a similar format to the IEEE 754 formats, but the number of bits for each component in the format changes. For example, as the exponent of a number approaches 0, more bits are allocated to the fraction (or mantissa) component, and therefore a higher precision is achieved. This has attracted many ML applications [5]–[7].

Typically, FP arithmetic is performed by a specific unit called the FP unit (FPU) in a general purpose central processing unit (CPU) core. It is well-known that FPUs take up much chip area and power [8], and many studies have tackled this problem [9]–[15]. One of the solutions is *approximate* FP arithmetic [11]–[15], which is based on the fact that there is

no need to waste energy to reach 100% accuracy as it is not always required [16]. Many applications in multimedia and ML fields have adopted this concept [16], [17].

In many applications, multiplication is one of the most frequently used but expensive operations. Therefore, there have been a lot of studies on custom hardware accelerators of approximate multipliers [11]–[15], [17]–[19]. For FP multiplication, a popular approach is to approximate mantissa multiplication [11]–[13]. For instance, Zhang *et al.* apply Mitchell's approximate multiplication algorithm [20] to mantissa multiplication [11]. Imani *et al.* discard some of mantissa depending on the magnitude of the mantissa [13]. These methods reduce the hardware area and/or computations at the expense of accuracy but cannot completely or granularly control the size of approximation errors. Another approach, called *iterative* approach, reuses small hardware to compensate for approximation errors [15], [18]. Since the accuracy improves as the number of iterations increases, an appropriate point between latency and accuracy, which is requested by an application, can be determined at runtime.

Compared to FP multipliers, there are few studies on posit multipliers. The existing studies aim at accurate calculations except for rounding [8], [21]–[23]. To the best of the authors' knowledge, this paper is the first work that proposes a hardware architecture of an *approximate* and *iterative* posit multiplier.

II. BACKGROUND

A. Posit Number System

The format of a posit is basically similar to the IEEE 754 FP formats [1], but a new concept is added for a wider dynamic range [4]. Fig. 1 shows a generic posit format for an nb -bit number. The most significant bit (MSB) is used to indicate the sign of the number, denoted as s . Its value is 0 for a positive number and 1 for a negative number. The next component includes $(m+1)$ run-length bits called *regime bits*. If successive m -bits are zeros and an opposite bit 1 follows, numerical meaning k is set to $-m$. On the other hand, if successive m ones are terminated by a bit 0, k is set to $m-1$. The following es -bits represent the exponent of the number, denoted as e . The remaining bits are used to represent the fraction, $1.f$. The hidden bit is the same as in the IEEE 754 FP formats. With this posit format, a real number x is calculated as follows:

$$x = (-1)^s \times used^k \times 2^e \times (1.f) \quad (1)$$

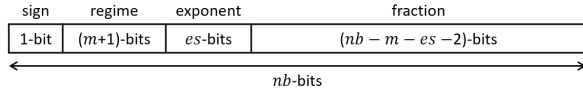


Fig. 1. A generic posit format for an nb -bit number [4].

where $useed = 2^{2^{es}}$. The regime and exponent bits together take the role of exponent bits in the IEEE 754 FP formats. However, there is no bias because the value of k can be negative. There are two exceptional cases: If all bits of the format are 0s, the value of x is 0; If MSB is 1 and all other bits are 0, the value of x is ∞ .

B. Iterative Multiplication

Mitchell's approximate multiplication algorithm is based on characteristics of a logarithm [20]. For an integer x , its logarithm is represented as follows:

$$\log_2 x = l + \log_2(1 + r) \quad (2)$$

where l refers to an integer part of the logarithm and r is a real number satisfying $0 \leq r < 1$. Using (2), multiplication between two integers, $x_1 (= 2^{l_1 + \log_2(1+r_1)})$ and $x_2 (= 2^{l_2 + \log_2(1+r_2)})$, is performed as follows:

$$x_1 \cdot x_2 = 2^{l_1 + l_2} \cdot (1 + r_1 + r_2 + r_1 \cdot r_2) \quad (3)$$

If the last term in parenthesis, $r_1 \cdot r_2$, is discarded, the multiplication can be replaced with shifters and adders. This approximate method is cheaper and faster than a conventional multiplication method but may generate significant approximation errors if the value of $r_1 \cdot r_2$ is large.

To compensate for the errors, Babic *et al.* repeat the same process to the discarded $r_1 \cdot r_2$ and add a new partial result to the first partial result [18]. This iterative process continues until a specific condition is met. If the partial result of i -th iteration is denoted as $P_i(x_1 \cdot x_2)$, the final multiplication result with n iterations is calculated as follows:

$$x_1 \cdot x_2 \approx \sum_{i=1}^n P_i(x_1 \cdot x_2) \quad (4)$$

As the value of n increases, the approximation error decreases while the delay to get the final result increases, which is trade-off. One of the biggest merits of the iterative method is that the value of n can be changed at runtime without additional circuits. On the other hand, unlike a fully pipelined method, it does not generate a result every cycle. However, interleaving between balanced operations is frequent in many applications, and therefore the iterative method can be effectively used [9].

Kim *et al.* apply the iterative approach to an SPFP number system [15]. Unlike a fixed integer format, an SPFP format has independent exponent and mantissa components, so there are separate circuits for each. In addition, only one of the two operands is approximated to reduce the size of the circuits, and various schemes such as *rounding scheme* and *operand selection scheme* are presented to reduce approximation errors. Our proposed work is based on this design but targets a posit number system.

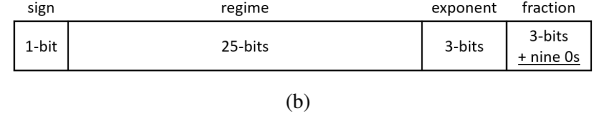
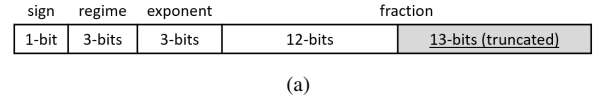


Fig. 2. Examples of variable truncation bits when the bit-width of operators for the fraction is set to 12. (a) 13 bits are truncated (b) Nine 0s are added.

III. APPROXIMATE AND ITERATIVE POSIT MULTIPLIER

A. Parameter Setting

One of the main goals of this work is to achieve similar accuracy to the previous approximate SPFP multiplier [15], while supporting a wider dynamic range. Therefore, the value of nb is set to 32. Meanwhile, the value of es is fixed at 3, which is recommended in the posit standard [4]. Suppose that $minpos$ and $maxpos$ refer to the smallest and largest positive numbers available, respectively. The recommended es value then provides $(minpos, maxpos)$ of $(6 \times 10^{-73}, 2 \times 10^{72})$ surpassing $(minpos, maxpos)$ of the IEEE SPFP format, $(1 \times 10^{-45}, 3 \times 10^{38})$. The remaining $(27-m)$ -bits of the posit format are used for the fraction component.

B. Variable Truncation Bits

Unlike the previous approximate SPFP multiplier design [15], keeping the number of truncation bits limits the functionality in posit multiplication because the number of bits in the fraction component varies depending on the number of bits in the regime component. Therefore, for the fraction component, the proposed design uses an operator with a fixed bit-width and changes the number of truncated bits. This bit-width is empirically chosen as 12. Fig. 2 shows two examples of the proposed method. The first example shown in Fig. 2(a) originally requires a 25-bit operator for the fraction component, but the least significant 13-bits are truncated to fit a 12-bit operator. On the other hand, in the second example shown in Fig. 2(b), nine 0s are added next to the least significant 3-bit fraction component, which does not incur truncation errors.

C. Iterative Architecture for Posit Multiplication

Fig. 3 shows a high-level block diagram of the proposed iterative posit multiplier architecture. The dotted lines represent stage registers. The sign of the final result is calculated as in the previous iterative SPFP multiplier design [15]. However, regimes of the two input operands are added together and exponents are added together as well, which differs from the previous design. The gray-colored parts, operand selector (OPS), shift amount calculator (SAC), and shift-based multiplier (SBM), are almost the same as those of the previous SPFP multiplier design. Fig. 4 shows an example when the proposed design multiplies two input posits X and Y . In this example, the input X is selected to be approximated by OPS, which is omitted in this figure. The sign of the output O ($SignO$) is

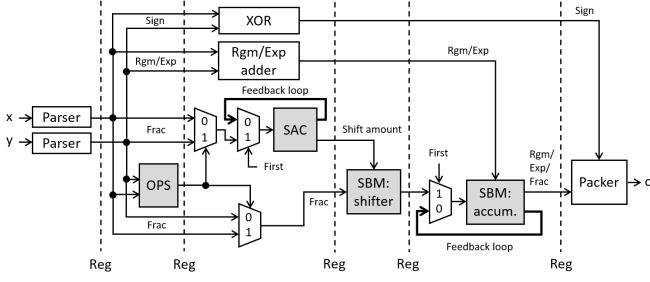


Fig. 3. High-level block diagram of the proposed posit multiplier architecture.

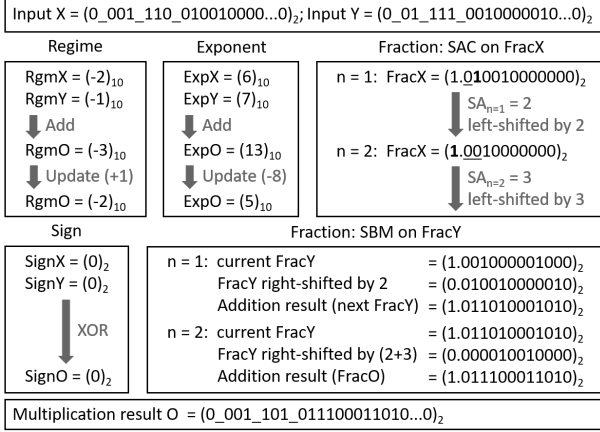


Fig. 4. An operation example of the proposed posit multiplier design.

calculated by performing the XOR operation. To calculate the regime of the output (RgmO), regimes of X and Y are added together. The exponent of the output (ExpO) is calculated in the same way. Since the initial ExpO is greater than 7, RgmO is incremented by 1 and ExpO is decremented by 8. The fraction of X (FracX) is put in SAC and generates a shift amount, which is the left-shift size required to move the next 1 of FracX to the hidden bit position, every cycle. In this example, only two 1s exist in FracX , and therefore the exact multiplication result is obtained after two iterations. Using the generated shift amounts, SBM shifts the fraction of Y (FracY) and accumulates it to generate the fraction of the output (FracO). After two iterations, the output posit O is generated by packing SignO , RgmO , ExpO , and FracO .

D. Parser and Packer

Since the number of bits for each component in the posit format varies, special parser and packer architectures are needed. Fig. 5(a) shows a block diagram of the proposed parser architecture. First, depending on the sign of an input posit, the original input or its two's complement is selected, which is implemented using XOR gates and an adder. The selected signal is represented as $t1$ in this figure. From $t1$, a lead one detector (LOD) and a lead zero detector (LZD) generate regime candidates. Since the value of nb is set to 32 in this work and one bit is used for the sign, these candidates are represented using 5 bits. Depending on MSB of $t1$, one of the two candidates is selected by a multiplexer. After the

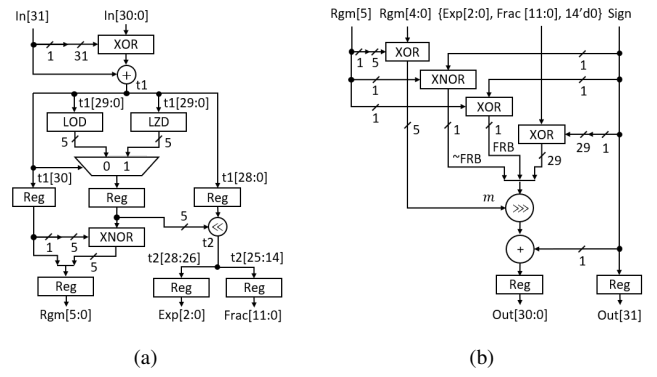


Fig. 5. Block diagrams of the proposed parser and packer architectures. (a) Parser (b) Packer.

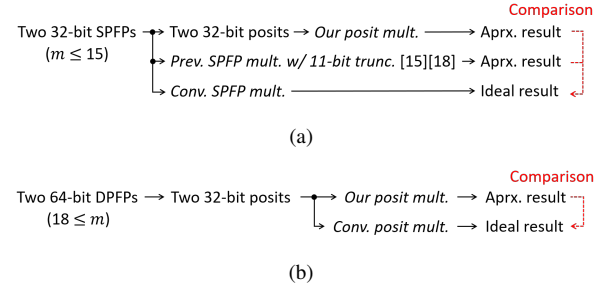


Fig. 6. Methodologies for accuracy comparison. (a) Comparison with the previous SPFP multipliers (b) Comparison with the conventional exact posit multiplier outside the range of SPFP numbers.

multiplexer, registers are deployed to cut the critical path delay. Using the selected regime, the final regime including the first regime bit (FRB) is generated. In the same cycle, the least significant 29 bits of $t1$ are logical-left-shifted by the selected regime value. The most significant 3 bits of the shifted result are used for the exponent, and the next bits are used for the fraction. As explained in Section III-B, the most significant 12 bits of the fraction are only used to reduce the number of hardware resources. Fig. 5(b) shows a block diagram of the proposed packer architecture. To fill the space for a regime in the posit format, the value of m is computed using regime bits of the multiplication result. Using this value, the concatenation of inverted FRB, original FRB, 3 exponent bits, 12 fraction bits, and 14 zero bits is arithmetic-right-shifted. The last adder in this figure is used for two's complement representation.

IV. EVALUATION

A. Accuracy

To evaluate the accuracy of the proposed design, two methodologies are used. First, to check if the proposed approximate posit multiplier can be an alternative to previous approximate SPFP multipliers, 32-bit posits are generated from SPFP numbers as shown in Fig. 6(a). For comparison, two approximate SPFP multipliers [15], [18] and a conventional SPFP multiplier are used. These SPFP multipliers directly receive two SPFP numbers. On the other hand, a format conversion to 32-bit posits is performed for the proposed posit multiplier. To generate neglectable errors in this format conversion, SPFP numbers that have m values of 15 or less are

TABLE I
ACCURACY WITHIN THE SPFP DYNAMIC RANGE

n	Algorithm	Correct rate			
		Err<0.1%	Err<0.5%	Err<1.0%	Err<5.0%
2	Babic [18] [†]	19.14%	47.37%	65.13%	99.13%
	Kim [15]	9.30%	32.12%	50.01%	95.58%
	Proposed	9.87%	32.19%	50.03%	95.57%
3	Babic [18] [†]	70.52%	95.52%	99.43%	100.00%
	Kim [15]	43.94%	83.01%	95.03%	100.00%
	Proposed	44.69%	83.17%	95.05%	99.99%
4	Babic [18] [†]	98.03%	100.00%	100.00%	100.00%
	Kim [15]	86.69%	99.81%	100.00%	100.00%
	Proposed	87.09%	99.79%	99.99%	99.99%

[†]This work is extended to SPFP multiplication as in [15].

TABLE II
ACCURACY OUTSIDE THE SPFP DYNAMIC RANGE

n	m range	Correct rate			
		Err<0.1%	Err<0.5%	Err<1.0%	Err<5.0%
2	over 22	92.13%	92.13%	92.13%	92.33%
	18–22	26.41%	31.17%	45.85%	95.56%
3	over 22	98.71%	98.71%	98.71%	98.76%
	18–22	68.96%	76.43%	89.87%	99.95%
4	over 22	99.73%	99.73%	99.73%	99.74%
	18–22	92.53%	95.69%	98.61%	99.95%

used. Since the previous approximate SPFP multipliers apply 11-bit truncation to the fraction, the difference in dynamic range, in which these m values are used, is insignificant. Second, as shown in Fig. 6(b), 32-bit posits are generated from DPFP numbers to see how accurate the proposed approximate posit multiplier is outside the dynamic range of SPFP numbers when compared to the conventional exact posit multiplier (that only has rounding errors). To avoid format conversion errors, the minimum m value is set to 18 in this experiment.

Table I shows the accuracy results when the test vector shown in Fig. 6(a) is used. The same approach as in [15], [18] is used to get the results: Approximation errors, denoted as Err, are calculated by $|ideal\ result - approximate\ result|/ideal\ result$ (%), and the percentage of 200 million multiplications satisfying a specific error bound, called *correct rate*, is obtained. Overall, Babic *et al.*'s work shows the highest accuracy because it applies the logarithm concept to both two input operands, which reduces errors but increases the number of hardware resources. However, when the value of n is 4, all works show correct rates higher than 86% for the Err<0.1% case. Meanwhile, if applications allow large approximation errors, n of 2 that shows the correct rate higher than 95% for the Err<5.0% case can be effectively used. The accuracy of the proposed work is almost the same as that of Kim *et al.*'s work [15]. This implies that the proposed approximate posit multiplier can be an alternative to the previous approximate SPFP multiplier for numbers with the SPFP dynamic range.

Table II shows the accuracy results when the test vector shown in Fig. 6(b) is used. If the value of m is larger than 22, the number of bits allocated to the fraction component is smaller than 6. Therefore, two iterations achieve more than 92% correct rates for the Err<0.1% case. On the other hand, if the value of m is smaller than 23, more iterations are required to reach the accuracy of a conventional posit multiplier.

TABLE III
COMPARISON OF FPGA IMPLEMENTATIONS

	Babic [18] [†]	Kim [15]	Jaiswal [23]	Proposed
Format	SPFP	SPFP	32-bit posit	32-bit posit
Architecture	Iterative	Iterative	Pipelined	Iterative
Device	xcvu190	xcvu190	xcvu9p	xcvu9p
f_{max} (MHz)	450	500	380	575
LUT	685	308	1,247	698
FF	241	100	303	289
Latency	$n+4$	$n+2$	5	$n+6$

[†] The results of Babic *et al.*'s work are brought from [15].

B. Hardware Implementation Results

This subsection evaluates FPGA implementations. Our design was implemented using Verilog HDL and synthesized and placed-and-routed using Xilinx Vivado Design Suite (2020.1). Our target device is Xilinx Virtex UltraScale+ FPGA (xcvu9p). Typically, multipliers are used in and/or with higher-level operators such as transforms. Since the higher-level operators often use a lot of digital signal processing (DSP) slices and on-chip memories, our design exploits only look-up tables (LUTs) and flip-flops (FFs) that are abundant on the FPGA.

Table III shows FPGA implementation results. Once again, the proposed design is compared with the two approximate and iterative SPFP multiplier designs. In addition, it is compared with a pipelined posit multiplier design [23]. For the implementation of the previous posit multiplier design, the authors' source code was used [24]. The default values of nb ($= 32$) and es ($= 6$) were used, while DSP slices used for fraction multiplication were replaced with LUTs and FFs for fair comparison. Compared with the SPFP multiplier designs, the proposed posit multiplier design requires more hardware resources and a longer delay. The majority of them originate from the parser and packer, which is common in conventional posit multiplier designs. Compared with the previous posit multiplier design, the proposed design reduces the number of LUTs by 44% thanks to the iterative approach while showing a 51% improvement in maximum frequency.

V. CONCLUSION

This paper presents the first approximate and iterative posit multiplier architecture. Generally, as more multiplier cores work in parallel at higher frequency, greater speedup is achieved. Therefore, the proposed multiplier design that is smaller and faster than the previous posit multiplier design can be effectively used in applications requiring a wide dynamic range of posit arithmetic. In future work, we will explore whether the approximate and iterative approach can be applied to custom hardware accelerators for a homomorphic encryption scheme supporting approximate arithmetic [25], [26].

ACKNOWLEDGMENT

This work was supported by the Institute for Information & Communications Technology Promotion (IITP) Grant through the Korean Government (MSIT), (Development and Library Implementation of Fully Homomorphic Machine Learning Algorithms supporting Neural Network Learning over Encrypted Data), under Grant 2020-0-00840.

REFERENCES

- [1] IEEE, "IEEE Standard for Floating-Point Arithmetic," *IEEE Std 754-2008*, pp. 1-58, Aug. 2008.
- [2] V. Sze, Y.-H. Chen, T.-J. Yang, and J. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295-2329, Dec. 2017.
- [3] J. L. Gustafson, *The End of Error: Unum Computing*, 1st ed. Boca Raton, FL, USA: CRC Press, 2015.
- [4] J. L. Gustafson and I. Yonemoto, "Beating Floating Point at its Own Game: Posit Arithmetic," *International Journal of Supercomputing Frontiers and Innovations*, vol. 4, no. 2, pp. 71-86, Jun. 2017.
- [5] Z. Carmichael, H. F. Langroudi, C. Khazanov, J. Lillie, J. L. Gustafson, and D. Kudithipudi, "Deep Positron: A Deep Neural Network Using the Posit Number System," in *Proc. DATE*, 2019.
- [6] J. Lu, S. Lu, Z. Wang, C. Fang, J. Lin, Z. Wang, and L. Du, "Training Deep Neural Networks using Posit Number System," in *Proc. SOCC*, 2019.
- [7] R. Murillo, A. A. Del Barrio, and G. Botella, "Deep PeNSieve: A Deep Learning Framework based on the Posit Number System," *Digital Signal Processing*, vol. 102, Jul. 2020.
- [8] R. Chaurasiya, J. Gustafson, R. Shrestha, J. Neudorfer, S. Nambiar, K. Niyogi, F. Merchant, and R. Leupers, "Parameterized Posit Arithmetic Hardware Generator," in *Proc. ICCD*, 2018.
- [9] H. Zhang, D. Chen, and S.-B. Ko, "Area-and Power-efficient Iterative Single/Double-precision Merged Floating-point Multiplier on FPGA," *IET Computers & Digital Techniques*, vol. 11, no. 4, pp. 149-158, Feb. 2017.
- [10] M. Al-Ashrafy, A. Salem, and W. Anis, "An Efficient Implementation of Floating Point Multiplier," in *Proc. SIEPC*, 2011.
- [11] H. Zhang, W. Zhang, and J. Lach, "A Low-Power Accuracy-Configurable Floating Point Multiplier," in *Proc. ICCD*, 2014.
- [12] J. Y. F. Tong, D. Nagle, and R. A. Rutenbar, "Reducing Power by Optimizing the Necessary Precision/Range of Floating-point Arithmetic," *IEEE Transactions on Very Large Scale Integration Systems*, vol. 8, no. 3, pp. 273-286, Jun. 2000.
- [13] M. Imani, D. Peroni, and T. Rosing, "CFPU: Configurable Floating Point Multiplier for Energy-Efficient Computing," in *Proc. DAC*, Jun. 2017.
- [14] P. Yin, C. Wang, W. Liu, and F. Lombardi, "Design and Performance Evaluation of Approximate Floating-Point Multipliers," in *Proc. ISVLSI*, Jul. 2016.
- [15] S. Kim and R. A. Rutenbar, "An Area-Efficient Iterative Single-Precision Floating-Point Multiplier Architecture for FPGA," in *Proc. GLSVLSI*, May 2019.
- [16] Z. Carmichael, H. F. Langroudi, C. Khazanov, J. Lillie, J. L. Gustafson, and D. Kudithipudi, "Is 'good enough' computing good enough?," *Communications of the ACM*, vol. 58, no. 5, pp. 12-14, Apr. 2015.
- [17] M. S. Kim, A. A. Del Barrio, R. Hermida, and N. Bagherzadeh, "Low-Power Implementation of Mitchell's Approximate Logarithmic Multiplication for Convolutional Neural Networks," in *Proc. ASP-DAC*, 2018.
- [18] Z. Babic, A. Avramovic, and P. Bulic, "An Iterative Logarithmic Multiplier," *Microprocessors and Microsystems*, vol. 35, no. 1, pp. 23-33, Feb. 2011.
- [19] S. Hashemi, R. Iris Bahar, and S. Reda, "DRUM: A Dynamic Range Unbiased Multiplier for Approximate Applications," in *Proc. ICCAD*, 2015.
- [20] J. N. Mitchell, "Computer Multiplication and Division Using Binary Logarithms," *IRE Transactions on Electronic Computers*, vol. EC-11, no. 4, pp. 512-517, Aug. 1962.
- [21] H. Zhang, J. He, and S.-B. Ko, "Efficient Posit Multiply-Accumulate Unit Generator for Deep Learning Applications," in *Proc. ISCAS*, 2019.
- [22] H. Zhang and S.-B. Ko, "Design of Power Efficient Posit Multiplier," *IEEE Transactions on Circuits and Systems-II: Express Briefs*, vol. 67, no. 5, pp. 861-865, May 2020.
- [23] M. K. Jaiswal, and H. K.-H. So, "PACoGen: A Hardware Posit Arithmetic Core Generator," *IEEE Access*, vol. 7, pp. 74586-74601, Jun. 2019.
- [24] "PACoGen: Posit Arithmetic Core Generator," [Online]. Available: <https://github.com/manish-kj/PACoGen/> (Accessed on 13 Sep, 2020)
- [25] S. Kim, K. Lee, W. Cho, J. H. Cheon, and R. A. Rutenbar, "FPGA-based Accelerators of Fully Pipelined Modular Multipliers for Homomorphic Encryption," in *Proc. ReConFig*, 2019.
- [26] S. Kim, K. Lee, W. Cho, Y. Nam, J. H. Cheon, and R. A. Rutenbar, "Hardware Architecture of a Number Theoretic Transform for a Bootstrappable RNS-based Homomorphic Encryption Scheme," in *Proc. FCCM*, 2020.